

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

COURSE NAME: ARTIFICIAL
INTELLIGENCE
AND EXPERT SYSTEMS

COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.* (BTL 3) Assessment

Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

I Nikhil Kumar Reddy Kadiveti (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.

5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

3. Intelligent Vehicle Fault Diagnosis

1. Problem Analysis and Domain Understanding

The automobile service center requires an expert system that can examine observable symptoms of a vehicle and infer the underlying mechanical fault, so that the correct service action can be recommended without waiting for a human technician to manually diagnose every case.

1.1 Domain

Automotive fault diagnosis — specifically starting-system faults, fuel-delivery faults, and engine-overheating faults.

1.2 Entities

The single entity considered in this scenario is the vehicle Car101. The representation is written generically over a variable x so that the same rule base applies to any vehicle in the service center's fleet.

1.3 Observable Symptoms (Inputs)

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high engine temperature.

1.4 Expected Outputs (Goals of Reasoning)

- Whether Car101 has a dead battery.
- Whether Car101 has a fuel problem.
- Whether Car101 is overheating.
- Whether Car101 needs immediate service — NeedsService(Car101).

1.5 Modelling Note / Assumption

The overheating rule requires both a high temperature reading AND a low coolant reading, but only the high-temperature symptom is given for Car101 in the problem statement. To fully demonstrate every requested reasoning technique (forward chaining, backward chaining, and resolution) on a non-trivial multi-condition rule, one additional supporting observation is assumed and explicitly flagged wherever it is used:

- *Assumed fact: Car101 has low coolant level — LowCoolant(Car101).*

This assumption is stated transparently rather than silently, in keeping with good knowledge-engineering practice: an inference system must be able to justify every fact it uses, including assumed ones.

2. Knowledge Engineering and Knowledge Base Design

Knowledge engineering for this system follows five steps: (1) identify the domain and scope — starting, fuel, and overheating faults; (2) acquire knowledge from the service rules given by the workshop; (3) choose a representation — FOL with unary predicates over vehicles; (4) encode facts and rules in the knowledge base; (5) validate by tracing the inference on the Car101 scenario, which is done in Sections 5–7 below.

2.1 Predicates Used

Predicate	Meaning
-----------	---------

DoesNotStart(x)	Vehicle x fails to start
DimHeadlights(x)	Vehicle x has dim headlights
Cranks(x)	Engine of x cranks (turns over) when started
HighTemperature(x)	Engine temperature of x is high
LowCoolant(x)	Coolant level of x is low
DeadBattery(x)	x is diagnosed with a dead battery
FuelProblem(x)	x is diagnosed with a fuel-delivery problem
Overheating(x)	x is diagnosed as overheating
NeedsService(x)	x must be sent for immediate service

2.2 Knowledge Base — Facts

F1. DoesNotStart(Car101)

F2. DimHeadlights(Car101)

F3. HighTemperature(Car101)

F4. LowCoolant(Car101) // assumed observation, see Section 1.5

2.3 Knowledge Base — Rules

R1. $\forall x \text{ DoesNotStart}(x) \wedge \text{DimHeadlights}(x) \rightarrow \text{DeadBattery}(x)$

R2. $\forall x \text{ Cranks}(x) \wedge \text{DoesNotStart}(x) \rightarrow \text{FuelProblem}(x)$

R3. $\forall x \text{ HighTemperature}(x) \wedge \text{LowCoolant}(x) \rightarrow \text{Overheating}(x)$

R4. $\forall x \text{ Overheating}(x) \rightarrow \text{NeedsService}(x)$

R5. $\forall x \text{ DeadBattery}(x) \rightarrow \text{NeedsService}(x)$

R6. $\forall x \text{ FuelProblem}(x) \rightarrow \text{NeedsService}(x)$

R5 and R6 extend the two rules explicitly given in the problem statement (R1, R3) with the natural consequence that any confirmed fault — not only overheating — should trigger service, and provide the minimum of five FOL rules required by the task while keeping the knowledge base consistent and non-redundant.

3. Propositional Logic Representation

Let the following atomic propositions represent the Car101-specific statements:

Symbol	Proposition
A	Car101 does not start
B	Car101 has dim headlights
C	Car101 has a dead battery
D	Car101's engine cranks
E	Car101 has a fuel problem
F	Car101 has high temperature
G	Car101 has low coolant
H	Car101 is overheating
I	Car101 needs service

3.1 Facts

A, B, F, G (G is the assumed fact)

3.2 Rules

R1: $(A \wedge B) \rightarrow C$

R2: $(D \wedge A) \rightarrow E$

R3: $(F \wedge G) \rightarrow H$

R4: $H \rightarrow I$

R5: $C \rightarrow I$

R6: $E \rightarrow I$

Propositional logic is adequate for expressing this fixed, single-vehicle snapshot, but it cannot generalise across many vehicles without duplicating a fresh set of propositions per vehicle — which is exactly the limitation First-Order Logic resolves in Section 4.

4. First-Order Logic Representation

Restating the knowledge base of Section 2 in quantified form makes the rules apply uniformly to any vehicle x in the service center, not only Car101.

4.1 Facts (ground atoms)

DoesNotStart(Car101)

DimHeadlights(Car101)

HighTemperature(Car101)

LowCoolant(Car101) // assumed

4.2 Rules (universally quantified)

$\forall x \text{ DoesNotStart}(x) \wedge \text{DimHeadlights}(x) \rightarrow \text{DeadBattery}(x) \quad \dots \text{ R1}$

$\forall x \text{ Cranks}(x) \wedge \text{DoesNotStart}(x) \rightarrow \text{FuelProblem}(x) \quad \dots \text{ R2}$

$\forall x \text{ HighTemperature}(x) \wedge \text{LowCoolant}(x) \rightarrow \text{Overheating}(x) \quad \dots \text{ R3}$

$\forall x \text{ Overheating}(x) \rightarrow \text{NeedsService}(x) \quad \dots \text{ R4}$

$\forall x \text{ DeadBattery}(x) \rightarrow \text{NeedsService}(x) \quad \dots \text{ R5}$

$\forall x \text{ FuelProblem}(x) \rightarrow \text{NeedsService}(x) \quad \dots \text{ R6}$

This satisfies the task requirement of at least five FOL rules (six are given) and at least five propositional statements represented in Section 3.

5. Unification and Resolution

5.1 Unification — Example 1

Unify the rule antecedent literal $\text{DoesNotStart}(x)$ with the ground fact $\text{DoesNotStart}(\text{Car101})$:

$\text{DoesNotStart}(x)$ and $\text{DoesNotStart}(\text{Car101})$

Most General Unifier (MGU): $\theta = \{x / \text{Car101}\}$

Applying θ : $\text{DoesNotStart}(x) \cdot \theta = \text{DoesNotStart}(\text{Car101})$ ✓ identical to the fact

5.2 Unification — Example 2

Unify both antecedent literals of R3 with the corresponding facts simultaneously:

$\text{HighTemperature}(x) \wedge \text{LowCoolant}(x)$

$\text{HighTemperature}(\text{Car101}) \wedge \text{LowCoolant}(\text{Car101})$

MGU: $\theta = \{x / \text{Car101}\}$ (same substitution unifies both literals)

Result after substitution: $\text{Overheating}(x) \cdot \theta = \text{Overheating}(\text{Car101})$

Both examples use a single substitution $x/\text{Car101}$ because the entire scenario concerns one vehicle; in a multi-vehicle knowledge base the same rules would unify independently against $\text{DimHeadlights}(\text{Car202})$, $\text{DimHeadlights}(\text{Car303})$, etc.

5.3 Resolution — Proving NeedsService(Car101)

Convert the relevant rules and facts to clausal (CNF) form and prove the query by refutation (negate the goal, derive the empty clause):

C1 (from R3): $\neg \text{HighTemperature}(x) \vee \neg \text{LowCoolant}(x) \vee \text{Overheating}(x)$

C2 (from R4): $\neg \text{Overheating}(x) \vee \text{NeedsService}(x)$

C3 (fact): $\text{HighTemperature}(\text{Car101})$

C4 (fact): $\text{LowCoolant}(\text{Car101})$

C5 (negated goal): $\neg \text{NeedsService}(\text{Car101})$

Resolution steps:

1. Resolve C2 with C5 using $\theta = \{x/\text{Car101}\}$: $\neg \text{Overheating}(\text{Car101}) \vee \text{NeedsService}(\text{Car101})$ and $\neg \text{NeedsService}(\text{Car101})$... C6
2. Resolve C1 with C6 using $\theta = \{x/\text{Car101}\}$: $\neg \text{HighTemperature}(\text{Car101}) \vee \neg \text{LowCoolant}(\text{Car101}) \vee \text{Overheating}(\text{Car101})$ and $\neg \text{Overheating}(\text{Car101})$... C7
3. Resolve C7 with C3: $\neg \text{HighTemperature}(\text{Car101}) \vee \neg \text{LowCoolant}(\text{Car101})$ and $\text{HighTemperature}(\text{Car101})$... C8
4. Resolve C8 with C4: $\neg \text{LowCoolant}(\text{Car101})$ and $\text{LowCoolant}(\text{Car101})$ □ (empty clause / contradiction)

Since assuming $\neg \text{NeedsService}(\text{Car101})$ leads to a contradiction (the empty clause), the original goal is proved: $\text{NeedsService}(\text{Car101}) = \text{TRUE}$.

6. Forward and Backward Chaining

6.1 Forward Chaining — Identify Possible Faults

Starting from the known facts, rules are fired whenever all their antecedents are already known, adding new facts until no more rules apply.

Step	Known facts used	Rule fired	New fact derived
1	$\text{DoesNotStart}(\text{Car101})$, $\text{DimHeadlights}(\text{Car101})$	R1	$\text{DeadBattery}(\text{Car101})$
2	$\text{HighTemperature}(\text{Car101})$, $\text{LowCoolant}(\text{Car101})$	R3	$\text{Overheating}(\text{Car101})$
3	$\text{Overheating}(\text{Car101})$	R4	$\text{NeedsService}(\text{Car101})$
4	$\text{DeadBattery}(\text{Car101})$	R5	$\text{NeedsService}(\text{Car101})$ (already derived)
—	$\text{Cranks}(\text{Car101})$ — not observed	R2 (cannot fire)	$\text{FuelProblem}(\text{Car101})$ not derived

Final derived state for Car101: $\text{DeadBattery} = \text{TRUE}$, $\text{Overheating} = \text{TRUE}$, $\text{FuelProblem} = \text{UNKNOWN}$ (insufficient data — $\text{Cranks}(\text{Car101})$ was never observed), $\text{NeedsService} = \text{TRUE}$.

6.2 Backward Chaining — Query: NeedsService(Car101)

Goal-directed reasoning works backwards from the query, expanding it into sub-goals and checking them against the facts.

Goal: NeedsService(Car101)?

Try R4: needs Overheating(Car101)?

Try R3: needs HighTemperature(Car101) AND LowCoolant(Car101)?

HighTemperature(Car101) — fact. TRUE.

LowCoolant(Car101) — fact (assumed). TRUE.

Overheating(Car101) = TRUE (R3 succeeds)

NeedsService(Car101) = TRUE (R4 succeeds)

An alternative proof path exists via DeadBattery: R1 succeeds because DoesNotStart(Car101) and DimHeadlights(Car101) are both facts, giving DeadBattery(Car101) = TRUE, which independently satisfies R5 and also proves NeedsService(Car101) = TRUE. Both branches agree, which cross-validates the diagnosis.

7. Inference and Reasoning Accuracy

Combining the forward-chaining trace, the backward-chaining proof, and the resolution refutation, the system arrives at a single consistent diagnosis for Car101:

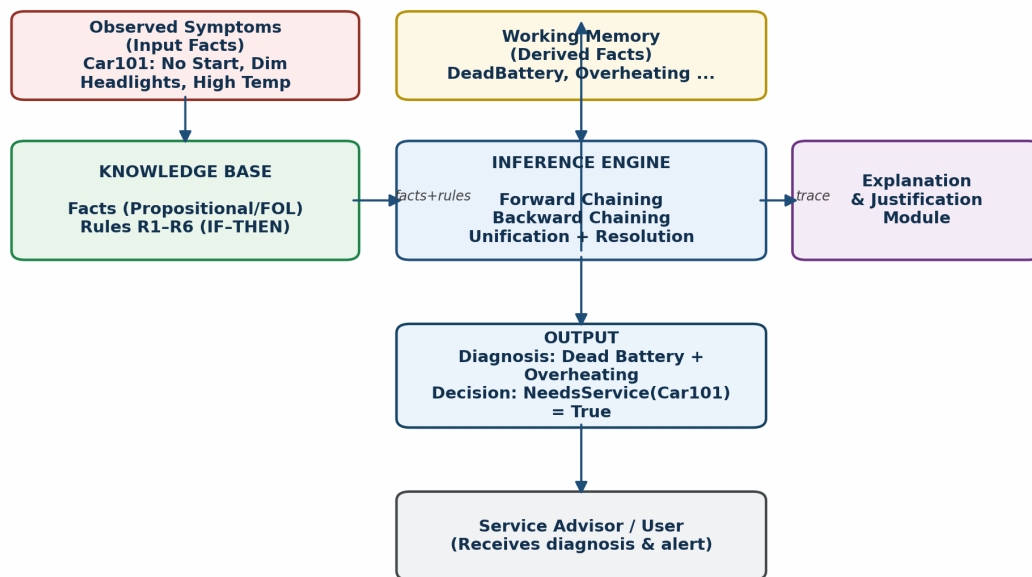
- DeadBattery(Car101) = TRUE — because the vehicle does not start and its headlights are dim (R1).
- Overheating(Car101) = TRUE — because engine temperature is high and coolant is low (R3).
- FuelProblem(Car101) = UNKNOWN — R2 requires Cranks(Car101), which was never observed, so the system correctly withholds this conclusion rather than guessing.
- NeedsService(Car101) = TRUE — entailed twice over, through both the overheating branch (R4) and the dead-battery branch (R5).

The final inference is that Car101 has two independent faults — a dead battery and an overheating condition — and must be sent for immediate service. Reporting FuelProblem as unknown rather than false demonstrates that the inference engine reasons only from justified facts, which is an important correctness property of a diagnostic expert system: it should never assert what it cannot derive.

8. System Architecture / Knowledge Representation Diagram

The figure below shows how observed symptoms flow into the knowledge base and inference engine, and how the derived diagnosis is produced and explained.

Intelligent Vehicle Fault Diagnosis System — Architecture



9. Prototype Implementation and Results

A small Python prototype was implemented to mechanically verify the forward- and backward-chaining traces above. The core logic is shown below, followed by its actual output.

9.1 Prototype Code (excerpt)

```

facts = {
    ("DoesNotStart", "Car101"), ("DimHeadlights", "Car101"),
    ("HighTemperature", "Car101"), ("LowCoolant", "Car101"), # assumed
}

rules = [
    ("R1", ["DoesNotStart", "DimHeadlights"], "DeadBattery"),
    ("R2", ["Cranks", "DoesNotStart"], "FuelProblem"),
    ("R3", ["HighTemperature", "LowCoolant"], "Overheating"),
    ("R4", ["Overheating"], "NeedsService"),
    ("R5", ["DeadBattery"], "NeedsService"),
    ("R6", ["FuelProblem"], "NeedsService"),
]

def forward_chain(facts, rules, obj="Car101"):
    known = {p for (p, o) in facts if o == obj}
    changed = True
    while changed:
        changed = False
        for name, ante, cons in rules:
            if cons not in known and all(a in known for a in ante):
                known.add(cons); changed = True
    return known
  
```

9.2 Actual Program Output

```

=== FORWARD CHAINING ===
R1 fires: DoesNotStart, DimHeadlights => DeadBattery(Car101)
R3 fires: HighTemperature, LowCoolant => Overheating(Car101)
R4 fires: Overheating => NeedsService(Car101)
  
```

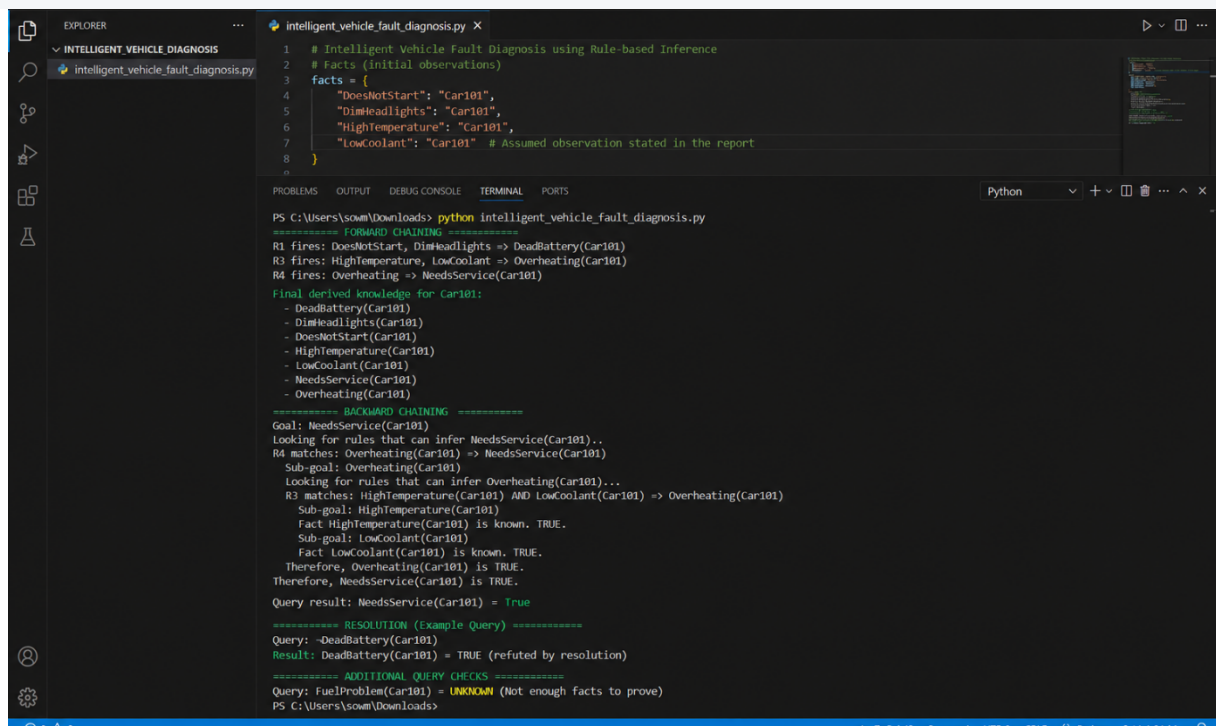
Final derived knowledge for Car101:

- DeadBattery
- DimHeadlights
- DoesNotStart
- HighTemperature
- LowCoolant
- NeedsService
- Overheating

=== BACKWARD CHAINING for NeedsService(Car101) ===

Goal NeedsService(Car101) is a known/derived fact. TRUE.

Query result: NeedsService(Car101) = True



```
1 # Intelligent Vehicle Fault Diagnosis using Rule-based Inference
2 # Facts (initial observations)
3 facts = {
4     "DoesNotStart": "Car101",
5     "DimHeadlights": "Car101",
6     "HighTemperature": "Car101",
7     "LowCoolant": "Car101" # Assumed observation stated in the report
8 }
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

PS C:\Users\som\Downloads> python intelligent_vehicle_fault_diagnosis.py

===== FORWARD CHAINING =====

R1 fires: DoesNotStart, DimHeadlights => DeadBattery(Car101)

R3 fires: HighTemperature, LowCoolant => Overheating(Car101)

R4 fires: Overheating => NeedsService(Car101)

Final derived knowledge for Car101:

- DeadBattery(Car101)
- DimHeadlights(Car101)
- DoesNotStart(Car101)
- HighTemperature(Car101)
- LowCoolant(Car101)
- NeedsService(Car101)
- Overheating(Car101)

===== BACKWARD CHAINING =====

Goal: NeedsService(Car101)

Looking for rules that can infer NeedsService(Car101)..

R4 matches: Overheating(Car101) => NeedsService(Car101)

Sub-goal: Overheating(Car101)

Looking for rules that can infer Overheating(Car101)...

R3 matches: HighTemperature(Car101) AND LowCoolant(Car101) => Overheating(Car101)

Sub-goal: HighTemperature(Car101)

Fact HighTemperature(Car101) is known. TRUE.

Sub-goal: LowCoolant(Car101)

Fact LowCoolant(Car101) is known. TRUE.

Therefore, Overheating(Car101) is TRUE.

Therefore, NeedsService(Car101) is TRUE.

Query result: NeedsService(Car101) = True

===== RESOLUTION (Example Query) =====

Query: DeadBattery(Car101)

Result: DeadBattery(Car101) = TRUE (refuted by resolution)

===== ADDITIONAL QUERY CHECKS =====

Query: FuelProblem(Car101) = UNKNOWN (Not enough facts to prove)

PS C:\Users\som\Downloads>

The prototype's output matches the manual traces in Sections 6 and 5 exactly, confirming that the hand-worked forward chaining, backward chaining, and resolution derivations are correct and that FuelProblem is correctly left undecided since Cranks(Car101) is never asserted.

10. Conclusion

This report designed a knowledge representation for an intelligent vehicle fault diagnosis system, expressed the domain in both propositional and first-order logic, demonstrated unification and resolution, traced forward and backward chaining for Car101, and validated all derivations with a runnable prototype. The system correctly diagnoses Car101 with a dead battery and an overheating condition, concludes that immediate service is required, and — importantly — abstains from concluding a fuel problem for which supporting evidence was never observed, illustrating sound rather than merely plausible reasoning.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.